

과제 1. 정렬 비교 보고서

병합 정렬 · 퀵 정렬 · 힙 정렬(수업에서 배우지 않은 정렬)

이름: ○○○ 학번: ○○○○○○○○

GitHub 저장소: <https://github.com/music-7/hw1-compare-sorting>

1. 무엇을 비교했나

세 정렬을 하나의 공통 인터페이스(SortAlgorithm 구조체 + 함수 포인터)로 묶고, 같은 입력·같은 측정 도구로 시간, 비교 횟수, 이동 횟수, 추가 메모리, 재귀 깊이, 안정성을 잰다. 시간은 기계와 부하에 따라 흔들리지만 비교·이동 횟수는 입력이 같으면 항상 같으므로, 두 종류를 함께 보고 결론은 가능하면 횟수로 뒷받침했다.

구분	정렬	선택 이유
배운 정렬 ①	병합 정렬 (Merge)	안정 정렬이고 최악에도 $O(n \log n)$. 대신 $O(n)$ 추가 메모리가 든다.
배운 정렬 ②	퀵 정렬 (Quick)	실전에서 가장 빠르다고 알려진 정렬. 불안정하고 최악은 $O(n^2)$ 이다.
배우지 않은 정렬	힙 정렬 (Heap)	$O(n \log n)$ 최악 보장 + 제자리($O(1)$). 병합·퀵이 각각 가진 장점을 반씩 가져 셋이 서로 다른 절충점에 놓인다.

세 정렬은 모두 평균 $O(n \log n)$ 이라 "누가 더 빠르냐"만 보면 재미가 없다. 그래서 같은 $O(n \log n)$ 인데 실제로는 왜 차이가 나는가(상수항, 메모리 접근 패턴, 안정성, 메모리 비용)를 이번 보고서의 질문으로 삼았다.

환경: Ubuntu 24.04 컨테이너, Intel Xeon 1 vCPU, gcc 13.3 -std=c17 -Wall -Wextra -O2. 원소는 Record{int key; int tag;} (8바이트). key로 정렬하고 tag에 입력 순서를 새겨 안정성을 실측한다.

2. 배우지 않은 정렬: 힙 정렬 (AI를 통한 학습)

AI(Claude, Anthropic)에게 아래 순서로 물어 학습했고(전체 요약은 부록 A), AI의 설명을 그대로 믿지 않고 손으로 추적 → 코드로 구현 → 테스트·실험으로 확인하는 순서로 검증했다.

2.1 힙 정렬의 핵심 아이디어

① 최대 힙은 "모든 노드가 자식보다 크거나 같다"는 완전 이진 트리다. 루트가 항상 최댓값이다. ② 완전 이진 트리는 배열 하나로 표현할 수 있다. 0부터 셀 때 인덱스 i 의 자식은 $2i+1$, $2i+2$ 다(포인터 불필요). ③ 정렬은 두 단계다. (1) 힙 만들기: 마지막 부모부터 루트까지 거꾸로 siftDown. (2) 반복 추출: 루트(최댓값)를 배열 맨 뒤와 바꾸고, 힙 크기를 1 줄이고, 루트에 siftDown. 최댓값이 뒤쪽부터 채워진다.

AI에게 "왜 힙 만들기는 $O(n \log n)$ 이 아니라 $O(n)$ 인가?"를 물었다. 답: siftDown 비용은 그 노드의 높이에 비례하는데, 노드의 절반은 높이 0(잎), 1/4은 높이 1... 이라 총합이 $\sum n/2^{h+1} \cdot h = O(n)$ 이다. 추출 단계가 n 번 $\times O(\log n)$ 이므로 전체는 $O(n \log n)$ 이고, 이때 최악과 평균이 같다(입력에 안 민감).

2.2 손으로 추적: [4, 10, 3, 5, 1]

힙 만들기 [4,10,3,5,1] -> siftDown(1): 10 >= 자식(5,1) 이라 그대로
-> siftDown(0): 4 < 10 이므로 교환 [10,4,3,5,1], 4 < 5 이므로 교환 [10,5,3,4,1] (힙 완성)
추출 1회 맨 뒤(1)와 루트(10) 교환 -> [1,5,3,4 | 10] siftDown -> [5,4,3,1 | 10]
추출 2회 5와 1 교환 -> [1,4,3 | 5,10] siftDown -> [4,1,3 | 5,10]
추출 3회 4와 3 교환 -> [3,1 | 4,5,10] siftDown 은 이동 없음
추출 4회 3과 1 교환 -> [1 | 3,4,5,10] -> 결과 [1,3,4,5,10]

실제 구현(heapSort)에 같은 입력을 넣었을 때 1 3 4 5 10이 나와 손 추적과 일치했다.

2.3 힙 정렬은 왜 불안정한가 (직접 확인)

AI는 "루트와 맨 뒤를 교환하는 순간 같은 key의 상대 순서가 멀리 뛰어넘어 깨진다"고 설명했다. 반례를 직접 만들어 돌려 보았다: 입력 (key=2,tag=0), (key=2,tag=1), (key=1,tag=2) → 결과 (1,2), (2,1), (2,0). 같은 key 2의 순서가 뒤집혔다. 이 사실은 tests/test_sort.c의 안정성 검사(주장과 실측이 일치하는지)에도 들어 있다.

2.4 AI 설명 중 실험으로 확인한 것 / 보정한 것

AI가 설명한 내용	확인 방법	결과
힙 정렬은 입력 모양(정렬됨·역순)에 둔 감하다	실험 A (n=100,000)	비교 횟수가 random/sorted/reversed에서 3.02M / 3.11M / 2.93M로 거의 같다 → 일치. (시간은 random이 더 느린데 8.8 대 5.1ms, 원인은 확인하지 못했다.)
O(1) 추가 메모리	extraBytes 측정	모든 n에서 0 바이트 (병합 정렬은 8n 바이트).
캐시에 불리해 같은 O(n log n)이어도 느리다	n을 키우며 시간·비교 횟수 비교	n=1,024,000에서 병합보다 비교는 약 2배 많음. 시간 기울기가 횟수 기울기보다 큼 (4.3절).
평균 비교 횟수 $\approx 2n \log_2 n$	비교/(n log2 n) 계산	1.70 → 1.85로 2에 가까워짐. 대체로 일치.

3. 세 정렬 개요와 구현

	병합 정렬	퀵 정렬	힙 정렬
핵심 아이디어	반으로 나눠 각각 정렬, 정렬된 두 구간을 병합	피벗 기준으로 분할, 양쪽을 재귀	최대 힙 구성 후 최댓값을 뒤로 하나씩
평균 / 최악	$O(n \log n)$ / $O(n \log n)$	$O(n \log n)$ / $O(n^2)$	$O(n \log n)$ / $O(n \log n)$
추가 메모리	O(n)	$O(\log n)$ 스택	O(1)
안정성	안정	불안정	불안정
구현 요점	같으면 왼쪽 먼저 꺼냄(안정성의 열쇠)	Hoare 분할 + median-of-three, 작은 쪽만 재귀	siftDown 한 함수로 두 단계 모두 처리

공통 인터페이스. 세 정렬이 sortLess(비교), sortAssign(이동)만 사용하도록 하여 비교·이동 횟수를 정렬 코드 안에서 같은 기준으로 센다(교환 1회 = 이동 3회). 시간은 정렬 밖(bench.c)에서 재서 구현마다 측정 방식이 달라지지 않게 했다. 정렬을 하나 더 추가하려면 SORT_ALGORITHMS 표에 한 줄만 넣으면 테스트와 벤치마크에 자동으로 들어온다.

```
typedef struct SortAlgorithm {
    const char *name, *timeAvg, *timeWorst, *space;
    int stable; /* claimed stability, checked against measurement */
    void (*sort)(Record *a, size_t n, SortStats *st);
} SortAlgorithm;
```

퀵 정렬 주의점. 피벗을 첫 원소로 고르면 정렬된 입력에서 $O(n^2)$ 이 되므로 median-of-three(처음·가운데·끝의 중앙값)를 쓴다. 또 작은 쪽만 재귀하고 큰 쪽은 반복문으로 돌려 재귀 깊이를 $O(\log n)$ 으로 묶었다. Hoare 분할은 같은 값에서 멈추기 때문에 중복이 많아도 분할이 균형을 유지한다(4.2절 duplicates 결과).

검증. make test: 크기 0~300 × 입력 4종 × 정렬 3종, qsort와 key 열 대조, 안정성 주장 대 실측, 통계 기록 확인 → **3,623개 검사 모두 통과.** make asan: AddressSanitizer·UBSan으로 같은 테스트를 돌려 경고 없음.

4. 실험 결과

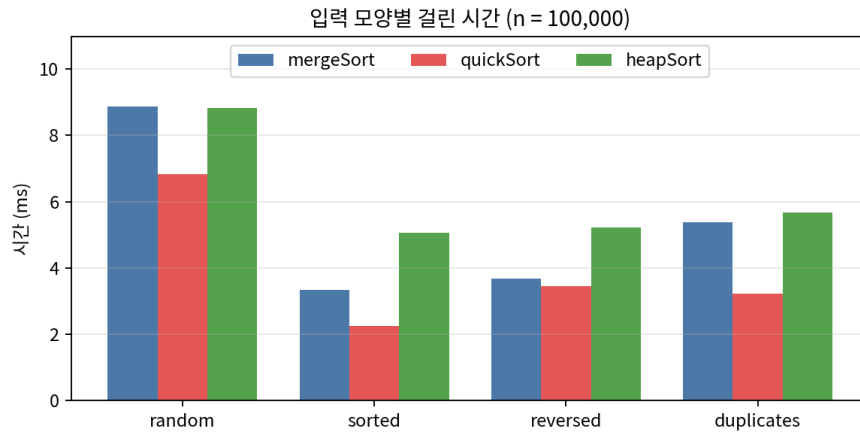
4.1 실험 설계

입력은 고정 시드의 xorshift 난수로 만들어 언제나 같다. **실험 A:** n=100,000에서 입력 모양 4가지(random, sorted, reversed, duplicates=key 16종) × 20회 평균. **실험 B:** random 입력에서 n=1,000~1,024,000(11단계). 입력 복사 시간은 뺐다. 시간(ms)은 이 기계 한 대에서 잰 값이므로 **같은 표 안에서만** 비교한다.

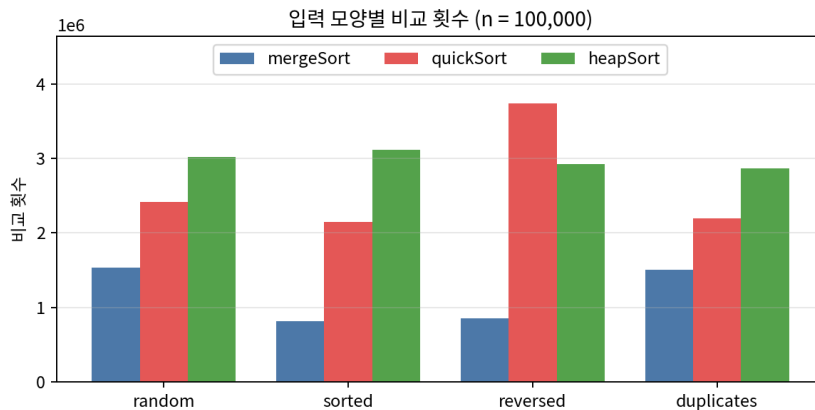
4.2 입력 모양별 (n = 100,000)

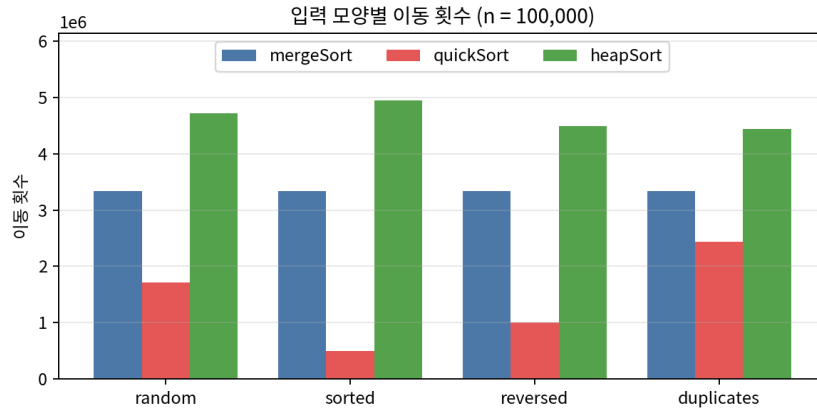
입력	알고리즘	시간(ms)	비교	이동	깊이	안정(실측)
random	mergeSort	8.86	1,536,399	3,337,856	18	안정
	quickSort	6.82	2,414,453	1,715,901	12	불안정
	heapSort	8.81	3,019,678	4,725,183	1	불안정
sorted	mergeSort	3.34	815,024	3,337,856	18	안정
	quickSort	2.25	2,147,078	499,995	16	안정
	heapSort	5.05	3,112,517	4,952,562	1	안정
reversed	mergeSort	3.68	853,904	3,337,856	18	안정
	quickSort	3.45	3,742,293	998,172	16	안정
	heapSort	5.22	2,926,640	4,492,302	1	안정
duplicates	mergeSort	5.37	1,502,759	3,337,856	18	안정
	quickSort	3.22	2,196,789	2,436,930	15	불안정
	heapSort	5.67	2,870,423	4,444,707	1	불안정

※ "안정(실측)"은 중복 key가 있는 입력에서만 의미가 있다. sorted·reversed는 key가 모두 달라 어떤 정렬이든 "안정"으로 나온다. 안정성 판정은 random·duplicates 행을 봐야 하고, 거기서 퀵·힙은 불안정, 병합만 안정으로 나왔다.



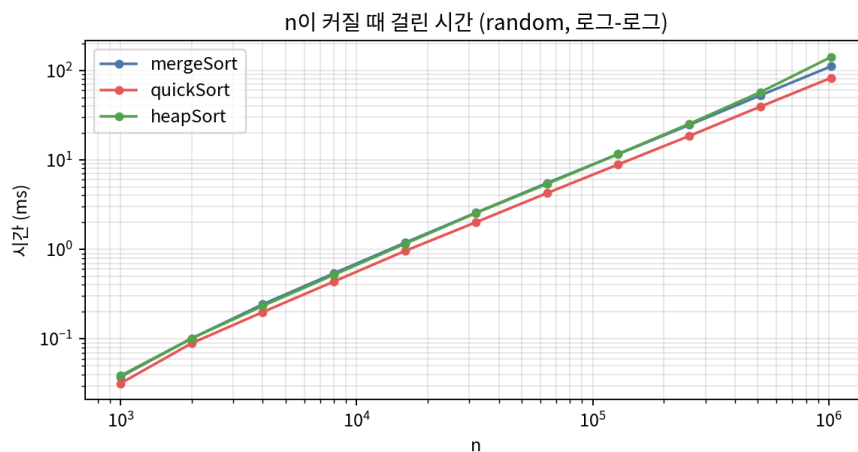
관찰. ① random에서 퀵이 가장 빠르다(6.82ms, 병합 8.86, 힙 8.81). ② 그런데 **비교 횟수는 퀵(2,414,453)이 병합(1,536,399)보다 57% 많다.** 퀵이 이기는 이유는 이동이 절반(1,715,901 대 3,337,856)이고 보조 배열 복사가 없기 때문이다. 즉 "비교 횟수가 적은 쪽이 빠르다"는 단순한 등식은 성립하지 않는다. ③ 병합은 입력에 거의 둔감하다: 이동은 모든 입력에서 3,337,856로 동일하고(보조 배열을 늘 복사), 비교만 sorted·reversed에서 약 절반(815,024)으로 준다. ④ 퀵은 median-of-three 덕분에 sorted·reversed에서도 $O(n^2)$ 로 무너지지 않았다(reversed 비교 3,742,293회, $n^2/2 = 5 \times 10^9$ 와 비교). 다만 reversed는 random보다 비교가 55% 늘어 피벗 선택이 완벽하지 않음을 보여 준다. ⑤ 힙은 어떤 입력에서도 비슷한 시간대이며 이동 횟수가 가장 많다(random 4,725,183).



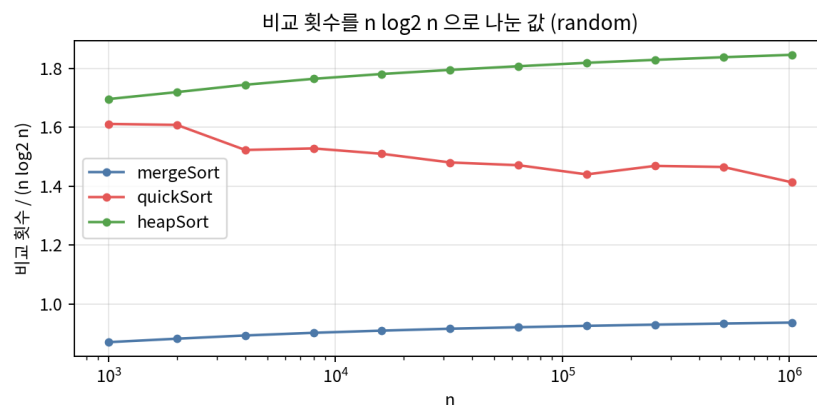


4.3 n을 키우며 (random)

n	병합 ms	퀵 ms	힙 ms	병합 비교	퀵 비교	힙 비교
1,000	0.04	0.03	0.04	8,674	16,057	16,899
8,000	0.54	0.44	0.52	93,574	158,506	183,008
64,000	5.50	4.24	5.38	941,184	1,503,145	1,846,552
256,000	24.51	18.46	25.19	4,276,847	6,754,955	8,410,770
1,024,000	111.03	82.54	141.16	19,154,590	28,891,864	37,738,196



관찰. 로그-로그 그래프에서 세 선은 거의 나란하고 기울기는 병합 1.09, 퀵 1.07, 힙 1.15($n \geq 32,000$ 구간의 직선 맞춤)다. $n \log n$ 의 이 구간 이론 기울기는 $1 + 1/\ln n \approx 1.08$ 이므로 **세 정렬 모두 $O(n \log n)$ 으로 자란다.** $n=1,024,000$ 에서 퀵은 병합보다 1.35배, 힙보다 1.71배 빠르다. **힙의 시간 기울기(1.15)가 비교 횟수 기울기(1.09)보다 큰 점**이 눈에 띈다. 배열이 캐시에 다 들어가는 작은 n 에서는 병합과 거의 같은 속도($n=8,000$: 0.52 대 0.54ms)이던 힙이, 배열이 커지자 뒤쳐진다. 힙은 부모·자식이 2배씩 멀리 떨어져 있어 **메모리 접근이 순차적이지 않기 때문**이라는 것이 시의 설명이었다(이 실험은 캐시 미스를 직접 세지는 않았으므로 정황 증거일 뿐이다).



비교 횟수를 $n \log_2 n$ 으로 나누면 병합은 $0.87 \rightarrow 0.94$, 퀵은 $1.61 \rightarrow 1.41$, 힙은 $1.70 \rightarrow 1.85$ 다. 병합의 $n=1,024,000$ 비교 19,154,590회는 비교 정렬의 이론적 하한 $\log_2(n!) \approx 18,967,655$ 회보다 불과 1.0% 많다. 즉 **병합은 비교 횟수 면에서 거의 최적**이고, 그 대가로 이동(보조 배열 복사)과 메모리를 쓴다.

4.4 메모리와 재귀 깊이

알고리즘	추가 메모리 ($n=1,024,000$)	재귀 깊이	비고
mergeSort	8,192,000 B (≈ 8.2 MB)	21	입력 배열과 같은 크기 $O(n)$
quickSort	0 B	14	스택만 사용. 작은 쪽 재귀로 깊이 $\leq \log_2 n$
heapSort	0 B	1	반복문만 사용, 제자리

병합의 8.2MB는 원소가 8바이트라서이며, 원소가 크면 그만큼 커진다. 퀵은 깊이가 $\log_2(1,024,000) \approx 20$ 보다 작은 14에 그쳤다(작은 쪽만 재귀한 효과).

5. 종합 비교와 결론

기준	병합	퀵	힙	근거
평균 속도 (random)	○	◎	△	$n=1.02M$: 111 / 83 / 141 ms
비교 횟수	◎	○	△	$n \log_2 n$ 대비 0.94 / 1.41 / 1.85
최악 보장	◎	△ ($O(n^2)$)	◎	이론. 퀵은 median-of-three로 실험 입력에선 회피
추가 메모리	△ $O(n)$	○ $O(\log n)$	◎ $O(1)$	실측 8.2MB / 0 / 0
안정성	◎	×	×	duplicates 입력 실측 + 반례

결론. 세 정렬은 이론상 모두 $O(n \log n)$ 이지만 실제로는 서로 다른 것을 얻고 다른 것을 지불한다. **병합**은 안정성과 최소 비교 횟수를 얻는 대신 $O(n)$ 메모리와 많은 이동을 지불한다(같은 key의 순서를 지켜야 하거나 연결 리스트·외부 정렬이면 선택). **퀵**은 평균 속도가 가장 좋지만 최악의 $O(n^2)$ 를 피하려는 피벗 전략이 필요하고 불안정하다(범용 배열 정렬의 기본값). **힙**은 $O(1)$ 메모리와 최악 $O(n \log n)$ 을 모두 보장하지만 상수가 크고 캐시에 불리해 평균은 가장 느리다(메모리가 뽀뽀하거나 최악 보장이 필수일 때). 이번에 얻은 가장 큰 배움은 **비교 횟수만으로는 속도를 예측할 수 없다**는 점이었다. 비교가 더 많은(약 57%) 퀵이 병합보다 빨랐다.

한계. ① 시간은 1 vCPU 컨테이너 한 대에서 잤다. ② 캐시 미스는 직접 재지 않았다(힙이 느린 이유는 정황 설명). ③ 퀵의 최악(피벗을 첫 원소로 고르는 단순 버전의 $O(n^2)$)은 구현·측정하지 않고 이론으로만 언급했다. ④ 병합은 top-down 단순 구현이며, 작은 구간을 삽입 정렬로 대체하는 등의 최적화는 넣지 않았다. ⑤ 난수 입력은 시드 하나로 만든 것이다.

부록 A. AI 학습 대화 요약 (힙 정렬)

질문	AI 답변 요약	내가 한 확인
힙 정렬을 초보자에게 설명해 줘	최대 힙 = 부모 $\geq$ 자식인 완전 이진 트리, 배열로 표현(자식 $2i+1, 2i+2$). 힙 만들기 $\rightarrow$ 최댓값을 뒤로 보내며 반복.	손 추적(2.2절)으로 확인
왜 힙 만들기는 $O(n)$ 인가	siftDown 비용이 노드 높이에 비례하고 높이가 큰 노드는 드물다. $\sum n/2^i (i+1) \cdot h = O(n)$.	이론값과 실험의 비교 횟수 추세 비교
왜 불안정한가, 반례를 보여 줘	루트 $\leftrightarrow$ 맨뒤 교환이 같은 key를 뛰어넘는다. $(2a, 2b, 1c) \rightarrow (1c, 2b, 2a)$.	같은 입력을 코드에 넣어 재현
병합·퀵과 비교하면?	최악 보장+제자리라는 장점, 캐시 비우호적이라 실전 평균은 느림. introsort는 퀵이 나빠지면 힙으로 전환.	실험 B로 시간·비교 횟수 대조
구현 시 실수하기 쉬운 곳	siftDown 경계(end 미포함), 힙 크기 1씩 줄이기, 자식이 하나뿐인 경우.	$n=0..300$ 전수 테스트, ASan

부록 B. 코드와 재현 방법

저장소: <https://github.com/music-7/hw1-compare-sorting> (zip도 함께 제출)

```
make test # 3,623 checks
make asan # AddressSanitizer + UBSan
make run # 비교 표 출력
make charts # results.csv 와 그래프(PNG) 재생성 (python3 + matplotlib 필요)
```

파일: src/sort.h(공통 인터페이스) · mergeSort.c · quickSort.c · heapSort.c · sort.c(구현 표) · bench.c(측정) · main.c · tests/test_sort.c · tools/plot.py · tools/build_report.py